

Concepte Cheie pentru Examen

Procese *Zombie* si *Orfane*

Sisteme de Operare - Tompea Maria-Daria

1 De ce sunt periculoase aceste concepte la examen?

Profesorii **adora** sa puna intrebari de tip capcana despre ce se intampla cand `exit()` si `wait()` *nu sunt sincronizate corect*. Daca nu stii exact ce se intampla, vei pierde puncte la intrebari aparent simple.

Regula de aur este simpla:

“Copilul apeleaza `exit()`, Parintele apeleaza `wait()`.”

Daca aceasta sincronizare lipseste, obtii fie un **Zombie**, fie un **Orfan**.

2 Recapitulare rapida: `fork()`, `exit()`, `wait()`

Inainte sa intelegem *Zombie* si *Orfani*, trebuie sa stim exact ce fac aceste functii.

2.1 `fork()` – Creaza un proces copil

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main() {
5     pid_t pid = fork(); // <-- aici se "duplica" procesul
6
7     if (pid < 0) {
8         // fork() a esuat (eroare)
9         perror("fork failed");
10        return 1;
11    } else if (pid == 0) {
12        // Suntem in procesul COPIL
13        // pid == 0 inseamna "eu sunt copilul"
14        printf("Salut! Eu sunt copilul, PID-ul meu: %d\n", getpid());
15    } else {
16        // Suntem in procesul PARINTE
17        // pid > 0 contine PID-ul copilului
18        printf("Eu sunt parintele, am creat copilul cu PID: %d\n",
19        pid);
20    }
21    return 0;
22 }
```

Listing 1: Exemplu de baza cu fork()

Analogie din viata reala

Gandeste-te la `fork()` ca la o **fotocopiere** a procesului. Dupa `fork()`, exista doua procese identice care ruleaza acelasi cod, dar fiecare stie cine este (parinte sau copil) prin valoarea returnata.

2.2 `exit(status)` – Termina procesul

Cand un proces termina, el apeleaza `exit()` cu un **cod de iesire**:

- `exit(0)` $\Rightarrow$ totul a mers bine (succes)
- `exit(1)` (sau orice alt numar nenul) $\Rightarrow$ a aparut o eroare

Ce se intampla dupa `exit()`?

1. Resursele procesului (memorie, fisiere deschise etc.) sunt eliberate.
2. **Dar** “certificatul de deces” (codul de iesire + PID) *ramane in tabela de procese*.
3. Parintele trebuie sa preia acest certificat cu `wait()`.

2.3 `wait(&status)` – Parintele asteapta copilul

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main() {
7     pid_t pid = fork();
8
9     if (pid == 0) {
10         // COPIL: face ceva si iese
11         printf("Copilul lucreaza...\n");
12         sleep(2);
13         printf("Copilul a terminat.\n");
14         exit(0); // <-- copilul se termina cu succes
15     } else {
16         // PARINTE: asteapta copilul
17         int status;
18         printf("Parintele asteapta...\n");
19         wait(&status); // <-- se blocheaza pana copilul termina
20         printf("Parintele a primit confirmarea!\n");
21
22         if (WIFEXITED(status)) {
23             printf("Copilul a iesit cu codul: %d\n", WEXITSTATUS(
24                 status));
25         }
26         return 0;
27     }
```

Listing 2: Parinte care asteapta copilul sa termine

3 Scenariul A: Procesul Zombie

Scenariul A: Procesul Zombie

Definitie: Un proces *zombie* este un copil care s-a terminat (a apelat `exit()`), dar al carui parinte **nu a apelat niciodata** `wait()` pentru a-i prelua codul de iesire. Intrarea procesului ramane blocata in tabela de procese a kernel-ului.

3.1 Cum apare un Zombie?

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main() {
6     pid_t pid = fork();
7
8     if (pid == 0) {
9         // COPIL: se termina imediat
10        printf("Copilul se termina acum (PID: %d)\n", getpid());
11        exit(0);
12        // Copilul a murit, DAR parintele nu va apela wait()!
13    } else {
14        // PARINTE: ignora copilul si doarme 60 de secunde
15        printf("Parintele ignora copilul si continua...\n");
16        sleep(60); // In acest interval, copilul este ZOMBIE
17        // Dupa ce parintele termina, zombie-ul este curatat de init
18    }
19    return 0;
20 }
```

Listing 3: Cod care creeaza un proces Zombie

Atentie!

Daca rulezi codul de mai sus si executi comanda `ps` aux in terminal in timpul celor 60 de secunde, vei vedea copilul cu statusul **Z** (Zombie):

```
user 1234 0.0 0.0 0 0 Z zombie_test <defunct>
```

Cuvantul `<defunct>` = procesul este mort dar inca "fantomeaza" in sistem.

3.2 De ce e o problema?

Fiecare proces, chiar si dupa moarte, **ocupa un slot in tabela de procese**. Aceasta tabela are o dimensiune maxima. Daca ai mii de zombie-uri:

- Tabela de procese se umple.
- Sistemul nu mai poate crea procese noi.

- Sistem crapat.

3.3 Fix: Parintele trebuie sa “recolteze” copilul

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main() {
7     pid_t pid = fork();
8
9     if (pid == 0) {
10         printf("Copilul se termina (PID: %d)\n", getpid());
11         exit(0);
12     } else {
13         int status;
14         // Parintele asteapta si "recolteaza" copilul
15         pid_t terminated = wait(&status);
16         printf("Parintele a curatat copilul cu PID: %d\n", terminated);
17         // Acum nu mai exista zombie!
18     }
19     return 0;
20 }
```

Listing 4: Versiunea corecta – fara Zombie

Fix / Solutie

Foloseste `waitpid(-1, &status, WNOHANG)` daca nu vrei ca parintele sa se blocheze si sa astepte. Flag-ul `WNOHANG` inseamna “verifica daca un copil a terminat, dar daca nu, nu te opri”.

4 Scenariul B: Procesul Orfan

Scenariul B: Procesul Orfan

Definitie: Un proces *orfan* este un copil care **inca ruleaza**, dar al carui parinte s-a terminat (sau a crapat) inainte ca el sa termine.

4.1 Cum apare un Orfan?

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main() {
6     pid_t pid = fork();
7
8     if (pid == 0) {
9         // COPIL: lucreaza indelung
10        printf("Copilul lucreaza (PID: %d)...\n", getpid());
```

```

11     sleep(30); // Copilul mai are treaba 30 de secunde
12     printf("Copilul a terminat.\n");
13     exit(0);
14 } else {
15     // PARINTE: se termina imediat, fara sa astepte copilul
16     printf("Parintele pleaca acasa (PID: %d). Pa copile!\n",
getpid());
17     exit(0); // <-- Parintele moare, copilul devine ORFAN
18 }
19 }

```

Listing 5: Cod care creeaza un proces Orfan

4.2 Ce se intampla cu orfanul?

Sistemul de operare **nu lasa procesele sa umble fara parinte**. Kernelul *adopteaza automat* orfanul:

Parintele orfanului devine Procesul 1 (init / systemd)

init este “tatal tuturor proceselor” si va apela automat `wait()` cand orfanul termina, curatand sistemul corect.

Analogie din viata reala

Imagineaza-ti ca esti un copil (proces) si parintii tai (procesul parinte) au plecat fara tine. Statul (kernelul) te trimite la un centru de plasament (init/systemd) care are grija de tine pana termini ce ai de facut.

Fix / Solutie

Spre deosebire de Zombie, **orfantul nu e neaparat o problema** – kernelul il gestioneaza automat. Totusi, poate fi nedorit daca procesul copil face ceva important si nu vrei sa ruleze “abandonat”.

5 Comparatie: Zombie vs. Orfan

Caracteristica	Zombie	Orfan
Cine a murit?	Copilul a apelat <code>exit()</code>	Parintele a apelat <code>exit()</code>
Cine traieste?	Parintele traieste (dar ignora)	Copilul traieste si lucreaza
Problema?	DA – ocupa tabela de procese	NU de obicei – kernelul il adopta
Cine curata?	Parintele cu <code>wait()</code>	init/systemd automat
Cum il recunosti?	Status Z in ps aux, <defunct>	PPID devine 1
Fix?	Parintele sa apeleze <code>wait()</code>	De obicei nu necesita fix

6 Comparatie: exit() vs. wait()

Caracteristica	exit(status)	wait(&status)
Cine il apeleaza?	Copilul (de obicei)	Parintele
Ce face?	Termina procesul curent	Pauzeaza parintele pana moare un copil
Ce argument primeste?	Un intreg (codul de iesire); 0 = succes	Un pointer unde se stocheaza statusul copilului
Se blocheaza?	Nu – nu mai returneaza nicio data	Da – “ingheata” pana moare un copil
Ce se intampla daca uiti?	Copilul executa codul parintelui!	Lasi Zombie-uri in urma ta

7 Exemplu complet: totul corect

Iata un exemplu complet care face totul bine – fara zombie, fara probleme:

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main() {
7     printf("== Procesul PARINTE incepe (PID: %d) ==\n", getpid());
8
9     pid_t pid = fork();
10
11     if (pid < 0) {
12         // fork() a esuat
13         perror("Eroare la fork");
14         exit(1);
15     }
16
17     if (pid == 0) {
18         //          COPIL
19
20         printf("[COPIL] Am inceput, PID-ul meu: %d\n", getpid());
21         printf("[COPIL] Parintele meu are PID: %d\n", getppid());
22
23         // Simuleaza ceva munca
24         sleep(2);
25
26         printf("[COPIL] Am terminat treaba. La revedere!\n");
27         exit(42); // Iesim cu codul 42 (poate fi orice)
28     } else {
29         //          PARINTE
30
31         printf("[PARINTE] Am creat copilul cu PID: %d\n", pid);
32         printf("[PARINTE] Astept sa termine...\n");

```

```
32
33     int status;
34     pid_t child_pid = wait(&status); // Blocare pana copilul
    termina
35
36     if (WIFEXITED(status)) {
37         // Copilul a iesit normal (nu a fost omorat de un semnal)
38         int exit_code = WEXITSTATUS(status);
39         printf("[PARINTE] Copilul %d a iesit cu codul: %d\n",
40               child_pid, exit_code);
41     }
42
43     printf("[PARINTE] Totul curat, niciun zombie!\n");
44 }
45
46 return 0;
47 }
```

Listing 6: Exemplu complet si corect cu fork/exit/wait

Output asteptat:

```
1 === Procesul PARINTE incepe (PID: 5000) ===
2 [PARINTE] Am creat copilul cu PID: 5001
3 [PARINTE] Astept sa termine...
4 [COFIL] Am inceput, PID-ul meu: 5001
5 [COFIL] Parintele meu are PID: 5000
6 [COFIL] Am terminat treaba. La revedere!
7 [PARINTE] Copilul 5001 a iesit cu codul: 42
8 [PARINTE] Totul curat, niciun zombie!
```

Listing 7: Output la rulare

8 Rezumat final pentru examen

Reguli de retinut la examen

1. **Copilul** apeleaza `exit()`, **Parintele** apeleaza `wait()`.
2. **Zombie** = copil mort, parinte care nu a apelat `wait()`. Ocupa tabela de procese. *Este o problema.*
3. **Orfan** = copil viu, parintele a murit. Kernelul il adopta automat la `init`. *De obicei nu e o problema.*
4. `exit(0)` = succes; `exit(nenul)` = eroare.
5. `wait()` se blocheaza. `waitpid(..., WNOHANG)` nu se blocheaza.
6. Daca uiti `exit()` in copil, copilul va executa codul parintelui – bug nasol!

Mult succes la examen!